

TEMPO-BIAS: Political Bias Analysis in LLMs

Ran Lu Hasti Hosseinpour Abderrahmane Moujar Oloruntobi Olutola

Paris-Saclay University
Fairness in AI Course under Prof. Adrian Popescu

January 2026

Experiment Setup: Data Collection

- Data collection methodology and political entities.
- Sentence templates and stratification.
- (Add details from your slides or figures.)

Experiment Setup: Prompt Input and Results

- Prompt design and Target-Oriented Sentiment Classification (TSC).
- Example prompts and model responses.
- (Add details or paste figures from original slide.)

Experiment Setup: IC Calculation

- Prediction Inconsistency (IC) as bias metric.
- Entropy-based formulation over template-level label distributions.
- (Add formulas or details from original.)

Temporal Bias Evaluation

Objective

Analyze the evolution of political bias over time in Large Language Models.

Experimental principle

- Models evaluated across successive releases.
- Bias treated as a *temporal phenomenon*, not a static property.

Controlled components (fixed)

- Political entities
- Sentence templates
- Target-Oriented Sentiment Classification

Only changing variable

- Model version

Temporal Evaluation Pipeline

Stage 1

- Creates a comparable temporal signal of political bias.
- Ensures changes reflect model evolution only.

Stage 2

- Quantifies how bias changes, not just its magnitude.
- Separates:
 - the overall trend of bias over time
 - impact of alignment tuning (base vs. chat variants)
 - ecosystem-level behavior (different model families)

Stage 3

- Makes temporal patterns explicit and interpretable.
- Enables comparison:
 - across releases
 - across model variants
 - across families

Stage 4

- Supports causal and longitudinal conclusions:
 - bias decay vs. accretion
 - alignment mitigation vs. amplification
 - convergence vs. divergence

Pipeline Overview

How we run the pipeline

- 1 **Environment:** create/activate virtualenv, install `requirements.txt`.
- 2 **Config:** YAML config (`run_config.yaml`) — dataset path, model provider, prompt, metrics, output dir.
- 3 **Run:** `python -m tempo_bias.main --config run_config.yaml`.

High-level flow (in code)

`main.py` → `PipelineController(config)` → `controller.analyze()`

- Load dataset, build prompts, init model (OpenAI / HuggingFace / vLLM).
- `PipelineRunner` runs inference, computes metrics (e.g. IC), writes responses and metrics to `outputs/`.

Repository layout (`tempo_bias/`)

- `main.py` — entry point; parses `--config`, calls `PipelineController`.
- `config/` — load and validate YAML (dataset, model, prompt, metrics).
- `datasets/` — load CSV (entities, templates, sentiments).
- `prompts/` — build TSC prompts (instruction, few-shot, label space).
- `models/` — adapters: `openai`, `hf`, `vllm`.
- `metrics/` — e.g. `ic.py` (Prediction Inconsistency).
- `pipeline/` — `controller.py` (orchestration), `runner.py` (inference loop).
- `outputs/` — write `responses.csv`, `metrics.csv`.
- `ui/` — Gradio app for interactive runs.

Code Snippet: Running the Pipeline

```
# main.py
parser.add_argument("--config", required=True)
args = parser.parse_args()
controller = PipelineController(args.config)
controller.analyze()
```

```
# controller.analyze() does:
# 1. set_seed(); 2. DatasetLoader(path)
# 3. PromptBuilder(spec); 4. Model adapter (openai/hf/vllm)
# 5. OutputWriter(); 6. PredictionInconsistency()
# 7. PipelineRunner(dataset, prompt_builder, model, metrics, ...)
# 8. runner.run() -> inference + metrics -> write CSV
```

(Add figures, IC comparison, or temporal trend plots here.)

(Add discussion points or key findings here.)

- TEMPO-BIAS: a temporal framework for political bias in LLMs.
- Multi-stage pipeline: comparable signal → quantification → interpretation → causal conclusions.
- (Add takeaway messages and future work.)

Thank you.